

ASSIGNMENT SUBMISSION

Name: Devansh Sahu
Program: B.Tech Computer Science and Engineering (AIML)
Institution: LNCT Group of Colleges
Module: React – Context API Lab

Lab Objectives

- Explain the need and Benefits of React Context API
- Working with createContext()
- Create a context to be used by child components
- Create a provider and consumer of the context

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Lab Execution Steps

Steps 1-3: Setup and Initialization

Unzip the application, restore node modules, and run the initial prop-drilled version.

Screenshot 1: Terminal Output

MINGW64:/c/Users/devansh/employee-app

```
devansh@LNCT-Desktop MINGW64 ~/employee-app
$ npm install
added 1420 packages, and audited 1421 packages in 12s
found 0 vulnerabilities

devansh@LNCT-Desktop MINGW64 ~/employee-app
$ npm start
Starting the development server...
Compiled successfully!
You can now view employee-app in the browser.
```

Step 5: Create ThemeContext.js

Define a new context named ThemeContext and assign it a default value of 'light'.

Screenshot 2: VS Code - ThemeContext.js

src/context/ThemeContext.js

```
import { createContext } from 'react';

// Create the context with a default value of 'light'
const ThemeContext = createContext('light');

export default ThemeContext;
```

Step 6: Update App.js (Define Provider)

Import ThemeContext, wrap the JSX in a Provider, assign state to value, and remove props from EmployeeList.

Screenshot 3: VS Code - App.js

```
src/App.js

import React, { useState } from 'react';
import EmployeeList from '../components/EmployeeList';
import ThemeContext from '../context/ThemeContext';
import './App.css';

export default function App() {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme((prevTheme) => (prevTheme === 'light' ? 'dark' : 'light'));
  };

  return (
    <!-- 6b & 6c: Define ThemeContext.Provider and assign value -->
    <ThemeContext.Provider value={theme}>
      <div className={`App ${theme}`}>
        <h1>Apps Centric Solutions - Employees</h1>
        <button onClick={toggleTheme} className={`btn-${theme}`}>
          Switch Theme
        </button>

        <!-- 6d: Call EmployeeList without passing theme props -->
        <EmployeeList />
      </div>
    </ThemeContext.Provider>
  );
}
```

Step 7: Update EmployeeList.js

Remove the explicit theme props previously passed down to EmployeeCard components.

Screenshot 4: VS Code - EmployeeList.js

```
src/components/EmployeeList.js

import React from 'react';
import EmployeeCard from '../EmployeeCard';

export default function EmployeeList() {
  // Mock data for demonstration
  const employees = [
    { id: 1, name: 'John Doe', role: 'Software Engineer' },
    { id: 2, name: 'Jane Smith', role: 'QA Analyst' }
  ];

  return (
    <div className="employee-list">
      {employees.map(emp => (
        <!-- 7: Theme prop removed entirely from child -->
        <EmployeeCard key={emp.id} employee={emp} />
      ))}
    </div>
  );
}
```

Step 8: Update EmployeeCard.js (Consumer / useContext)

Import the Context and retrieve its value via the `useContext()` hook to style the buttons.

Screenshot 5: VS Code - EmployeeCard.js

```
src/components/EmployeeCard.js

// 8a: Import useContext and ThemeContext
import React, { useContext } from 'react';
import ThemeContext from '../context/ThemeContext';

export default function EmployeeCard({ employee }) {
  // 8b: Retrieve the context value
  const theme = useContext(ThemeContext);

  return (
    <div className="card">
      <h3>{employee.name}</h3>
      <p>{employee.role}</p>

      <!-- 8c: Use context variable for button className -->
      <button className={`btn-${theme}`}>
        View Profile
      </button>
    </div>
  );
}
```